SentientShield Project Flows

A consolidated PDF reference of the major runtime flowcharts used in the SentientShield-ML-Detector project.

1. High-level system architecture

Mermaid diagram definition:

```
flowchart LR
  U[User Browser] -->|HTTP| N[Nginx / Reverse Proxy\noptional]
  U -->|HTTP| API[FastAPI / Uvicorn\nsrc.main:app]
  N -->|/| API
  N -->|/dashboard or /analytics| ST[Streamlit]

  API -->|serves| STATIC[Static UI\npremium.html / app.js / styles.css]
  API --> ROUTERS[API Routers\n/api/* and /neuralfort/*]
  ST -->|fetch JSON| ROUTERS

  ROUTERS --> STORE[(SQLite / PostgreSQL\nTrendStore / PGTrendStore)]
  ROUTERS --> INTEL[(SQLite\nlogs/intelligence.db)]
  ROUTERS --> FILELOG[(JSONL logs\napi_calls.jsonl / perf.jsonl)]
```

2. Authentication flow (JWT)

Mermaid diagram definition:

```
sequenceDiagram
  participant UI as Browser UI / Streamlit
  participant API as FastAPI
  participant AUTH as auth_dependency

  UI->>API: Request with Authorization: Bearer JWT
  API->>AUTH: verify_jwt_token(token)
  AUTH-->>API: payload or 401
  API-->>UI: JSON response
```

3. Command Center UI to API request flow

Mermaid diagram definition:

```
sequenceDiagram
  participant User
  participant UI as premium.html + app.js
  participant API as FastAPI
  participant SVC as Service Layer

  User->>UI: Click button
  UI->>UI: Build payload + attach token
  UI->>API: POST / GET request
  API->>SVC: Run handler / ML logic
  SVC-->>API: Result
  API-->>UI: JSON response
  UI-->>User: Render output
```

4. Web-attack prediction flow

Mermaid diagram definition:

```
flowchart TD
  A[Client POST /api/predict] --> B[Load model + metadata]
  B --> C[Build feature row]
  C --> D[Run inference\npredict_proba or predict]
  D --> E{raw_log present?}
  E -- Yes --> F[Run intelligence pipeline]
  E -- No --> G[Write api_calls.jsonl]
  F --> G
  G --> H[Return JSON\npredicted_label, confidence, probabilities]
```

5. Log triage pipeline flow

Mermaid diagram definition:

```
flowchart TD
  A[Client POST /api/logs/pipeline/run] --> B[Initialize pipeline context]
  B --> C[Severity classification]
  C --> D[Threat type classification]
  D --> E[Embedding generation]
  E --> F[Optional vector ingest]
  F --> G[MITRE ATT&CK mapping]
  G --> H[Risk scoring]
  H --> I[Optional SOC report generation]
  I --> J[Return JSON result]
```

6. Event storage and analytics trends flow

Mermaid diagram definition:

```
sequenceDiagram
  participant UI as Streamlit / Command Center
  participant API as /api/logs/*
  participant DB as TrendStore / PGTrendStore
  participant MITRE as MITREAttckMapper

  UI->>API: POST /api/logs/record-event
  API->>API: Classify severity + threat_type
  API->>DB: record_event(message, severity, threat_type, risk)
  API->>MITRE: map(message)
  API-->>UI: severity, threat_type, risk

  UI->>API: GET /api/logs/trends/*
  API->>DB: Aggregate trend queries
  API-->>UI: Trend data for charts
```

7. Streamlit Threat Analytics dashboard flow

Mermaid diagram definition:

```
flowchart TD
  U[User opens Streamlit page] --> ST[Streamlit server]
  ST -->|requests.get / post| API[FastAPI backend]
  API -->|JSON response| ST
  ST -->|charts / tables / cards| U
```

8. Dashboard realtime websocket heartbeat flow

Mermaid diagram definition:

```
sequenceDiagram
  participant UI as Browser Dashboard
  participant WS as /api/ws/realtime

  UI->>WS: Open WebSocket
  loop every 2.5 seconds
    WS-->>UI: threat_update heartbeat message
  end
```

9. NeuralFort website analysis flow

Mermaid diagram definition:

```
flowchart TD
  A[Client POST /neuralfort/analyze-website] --> B[perform_advanced_website_analysis]
  B --> C[Calculate score, grade, threat categories]
  C --> D[Generate recommendations, risk level, ML confidence]
  D --> E[Write API log entry]
  E --> F[Return analysis JSON]
```

10. NeuralFort resilience framework lifecycle

Mermaid diagram definition:

```
flowchart TD
  A[Register / Activate website] --> B[get_neuralfort_framework]
  B --> C[Collect system metrics]
  C --> D[Update anomaly detector history]
  D --> E[Healing engine suggests / executes actions]
  E --> F[Expose via anomalies and healing endpoints]
```

11. Batch log processing workflow

Mermaid diagram definition:

```
flowchart TD
  A[POST /api/logs/workflow/batch-process] --> B{RENDER env var set?}
  B -- Yes --> C[RenderAsync.start_task]
  C --> D[Worker runs process_logs_task]
  D --> E[PipelineEngine.run for each log]
  E --> F[Return workflow_run_id or results]

  B -- No --> G[Run local process_logs_task]
  G --> E
```

12. Scheduled retraining flow

Mermaid diagram definition:

```
sequenceDiagram
  participant API as FastAPI Startup
  participant SCH as daily_retrain_loop
  participant RT as scripts.retrain.daily_retrain

  API->>SCH: start_daily_retrain_task()
  loop every retrain interval
    SCH->>RT: daily_retrain()
    RT-->>SCH: success / failure result
  end
```

13. Docker Compose runtime flow

Mermaid diagram definition:

```
flowchart LR
  DC[docker-compose up] --> API[sentientshield-api\nport 10000]
  DC --> ST[sentientshield-analytics\nport 8501]
  API --> V1[(docker/logs volume)]
  API --> V2[(docker/artifacts volume)]
```

14. HF Spaces Docker flow

Mermaid diagram definition:

```
flowchart TD
  P[Public Port 7860] --> N[Nginx]
  N -->|/| API[Uvicorn on 8000]
  N -->|/analytics/| ST[Streamlit on 8501]
```

15. Cloudflare Tunnel live demo flow

Mermaid diagram definition:

```
flowchart LR
  U[Remote device] --> CF[trycloudflare.com]
  CF --> C[cloudflared on local PC]
  C --> API[localhost:10000]
```